Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности

Высшая школа программной инженерии

ПРАКТИЧЕСКАЯ РАБОТА № 1

Фильтрация больших объемов данных с помощью вероятностной структуры данных Bloom Filter и классической HashSet

по дисциплине «Методы тестирования программного обеспечения»

Выполнил

студент гр.5130903/40003 Г.Н. Асадуллаев

Руководитель

старший преподаватель

ВШ ПИ В.А. Пархоменко

Санкт-Петербург – 2025

СОДЕРЖАНИЕ

Введение 4
1 АНАЛИЗ ЗАДАЧИ И ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ 6
1.1 Назначение и общая логика работы 6
1.2 Назначение и общая логика работы 7
1.3 Входные и выходные данные 7
1.4 Проектирование программной структуры 8
1.4.1 Структура программы 9
2 РЕАЛИЗАЦИЯ МОДУЛЯ И АЛГОРИТМЫ ФИЛЬТРАЦИИ 10
2.1 Модуль ввода/вывода и пакетной обработки JSON (com.ipfilter.service) 10
2.1.1 Методы класса JsonHandler 10
2.1.2 Методы класса FilterService 10
2.2 Методы модуля фильтрации (com.ipfilter.filter) 11
2.2.1 Методы интерфейса IpFilter 11
2.2.2 Методы класса HashSetFilter 12
2.2.3 Методы класса BloomFilter 12
2.3 Методы пользовательского интерфейса (com.ipfilter.Main) 13
2.3.1 Метод main 13
3 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ МОДУЛЬНЫХ ТЕСТОВ 14
3.1 Запуск Pitest и используемые настройки 14
3.2 Задействованные мутационные операторы 14
3.3 3.2 Задействованные мутационные операторы 14
3.4 Подробный анализ мутаций для участка кода 15
3.4.1 Метод contains (класс BloomFilter) 15
3.4.2 Метод processAndCompare (класс FilterService) 15
3.5 Эквивалентные мутации 16
3.6 Метрики мутационного тестирования и их интерпретация 16
3.7 Оценка mutation-adequate и улучшение качества тестов 16
4 МУТАЦИОННОЕ ТЕСТИРОВАНИЕ И ОЦЕНКА КАЧЕСТВА ТЕСТОВ 17
4.1 Операторы мутаций и примеры их применения в проекте 17
4.1.1 Пример мутации NEGATE_CONDITIONALS 17
4.1.2 Пример мутации CONDITIONALS_BOUNDARY 18
4.1.3 Пример мутации VOID_METHOD_CALLS 18
4.2 Эквивалентные мутации и особенности интерпретации результатов 19
4.3 Результаты мутационного тестирования 19

Заключение 20
Список использованных источников 22
Приложение 1 Программный код 23

ВВЕДЕНИЕ

Фильтрация больших объёмов данных является одной из наиболее распространённых задач при разработке высоконагруженных систем и сетевых приложений. Данная задача относится к классу задач оптимизации поиска и проверки принадлежности элементов к множеству: необходимо определить, содержится ли заданный IP-адрес в чёрном списке (blacklist). Проверка IP-адресов применяется практически во всех сетевых системах — от фаерволов и систем безопасности до веб-серверов и прокси-серверов. Некорректная или неэффективная реализация фильтрации может приводить к значительным задержкам в обработке запросов, повышенному потреблению памяти и снижению общей производительности системы.

Актуальность/мотивация работы обусловлена тем, что рост объёмов сетевого трафика и увеличение размеров чёрных списков IP-адресов (которые могут содержать миллионы записей) требуют применения эффективных структур данных и алгоритмов фильтрации. Поэтому помимо реализации самой логики фильтрации важно обеспечить её корректность с помощью модульного тестирования и сравнить производительность различных подходов. Дополнительным инструментом оценки качества тестового набора является мутационное тестирование, позволяющее определить, насколько тесты способны обнаруживать типовые дефекты в коде.

Цель работы — разработка Java-модуля для фильтрации IP-адресов с использованием двух структур данных (классический HashSet и вероятностная структура Bloom Filter), реализация консольного приложения для интерактивной и пакетной обработки данных, а также создание и анализ набора модульных тестов с использованием JUnit 5, Hamcrest, Mockito и PIT.

Определим **задачи работы**, которые мы должны будем выполнить в ходе разработке программного обеспечения:

A. Определить требования к фильтрации IP-адресов и формализовать правила работы с чёрным списком.
B. Спроектировать структуру модуля и реализовать два алгоритма фильтрации: детерминированный подход на основе HashSet и вероятностный подход на основе Bloom Filter.
C. Реализовать консольное приложение для интерактивной проверки IP–адресов, а также пакетную обработку входных данных из JSON-файла с формированием отчёта и рекомендациями по выбору алгоритма.

D. Разработать модульные тесты с применением различных техник (включая параметризованные тесты), а также использовать Hamcrest и Mockito для проверки результатов и взаимодействий.
E. Выполнить мутационное тестирование с помощью PIT, проанализировать мутационные операторы и метрики качества тестирования (Line Coverage, Mutation Coverage, Test Strength), сделать отчет, сформулировать выводы.

1 АНАЛИЗ ЗАДАЧИ И ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

Целью данной главы является описание задачи фильтрации IP-адресов по чёрному списку и обоснование принятых решений по структуре разрабатываемого программного продукта, организации ввода/вывода данных и реализуемому функционалу. В главе формулируются требования к проверке IP-адресов, описывается пользовательское взаимодействие с приложением (функционал меню, режимы работы) и приводятся основные сценарии использования (use cases) для интерактивной и пакетной проверки IP-адресов.

1.1 Назначение и общая логика работы

Разработанное консольное приложение предназначено для проверки принадлежности IP-адреса к чёрному списку (blacklist) по заданным правилам фильтрации. Программа поддерживает два режима работы: (1) интерактивную проверку IP–адресов, вводимых с клавиатуры, и (2) пакетную проверку набора IP-адресов, загружаемого из JSON-файла, с формированием отчёта и рекомендациями по выбору алгоритма. Основная логика работы приложения:

- Пользователь загружает чёрный список IP-адресов из JSON-файла
- Программа загружает данные в две структуры данных одновременно: HashSet и Bloom Filter
- Пользователь вводит IP-адрес для проверки или загружает файл со списком для проверки
- Программа проверяет IP-адрес обоими методами и сравнивает результаты
- Выводится результат проверки для каждого алгоритма отдельно
- Формируется статистика (память, время выполнения, вероятность ошибки для Bloom Filter)
- Генерируется рекомендация по выбору алгоритма для данного объёма данных

Ключевое отличие алгоритмов:

- HashSet — детерминированный алгоритм с 100% точностью, но высоким потреблением памяти (хранит все элементы полностью)
- Bloom Filter — вероятностный алгоритм с возможностью False Positive (ложное срабатывание), но невозможностью False Negative (ложное отрицание), при этом значительно экономит память

1.2 Назначение и общая логика работы

После запуска приложения пользователю отображается меню со следующими пунктами:

- Загрузить blacklist из JSON - Пользователь вводит путь к JSON-файлу с чёрным списком IP-адресов. Программа загружает данные в обе структуры данных (HashSet и Bloom Filter) и подтверждает успешную загрузку.
- Проверить IP адрес — Пользователь вводит IP-адрес для проверки. Программа выполняет проверку обоими методами и выводит результат для каждого алгоритма отдельно (FOUNDNOT_FOUND).
- Сравнить алгоритмы — Программа выполняет проверку заранее заданного набора IP—адресов, выводит сравнительную статистику (память, время, вероятность ошибки) и формирует рекомендацию по выбору алгоритма.
- Справка — Приложение выводит краткую информацию о назначении программы и принципах работы алгоритмов фильтрации.
- Выход — Завершение работы приложения.

Для более ясного понимания поведения программы приведем диаграмму случаев использования. Схема вкратце демонстрирует возможности пользователя, роль приложения относительно пользователя и как происходит взаимодействие с входными данными. ПО предоставляет пользователю гибкий интерфейс для работы с фильтрацией IP-адресов, позволяя как проверять отдельные адреса в интерактивном режиме, так и обрабатывать большие наборы данных в пакетном режиме с формированием детального отчёта.

1.3 Входные и выходные данные

Интерактивный режим принимает на вход путь к JSON-файлу с чёрным списком IP-адресов и отдельный IP-адрес для проверки (строка формата IPv4) Пакетный режим принимает на вход JSON-файл произвольного размера, содержащий:

- blacklist — массив IP-адресов для добавления в чёрный список
- checkIps — массив IP-адресов для проверки

На выходе для интерактивного режима формируется статус проверки для каждого алгоритма (FOUNDNOT_FOUND) в консоль, тогда как для пакетного режима JSON-отчёт с полями: timestamp — время выполнения проверки, results

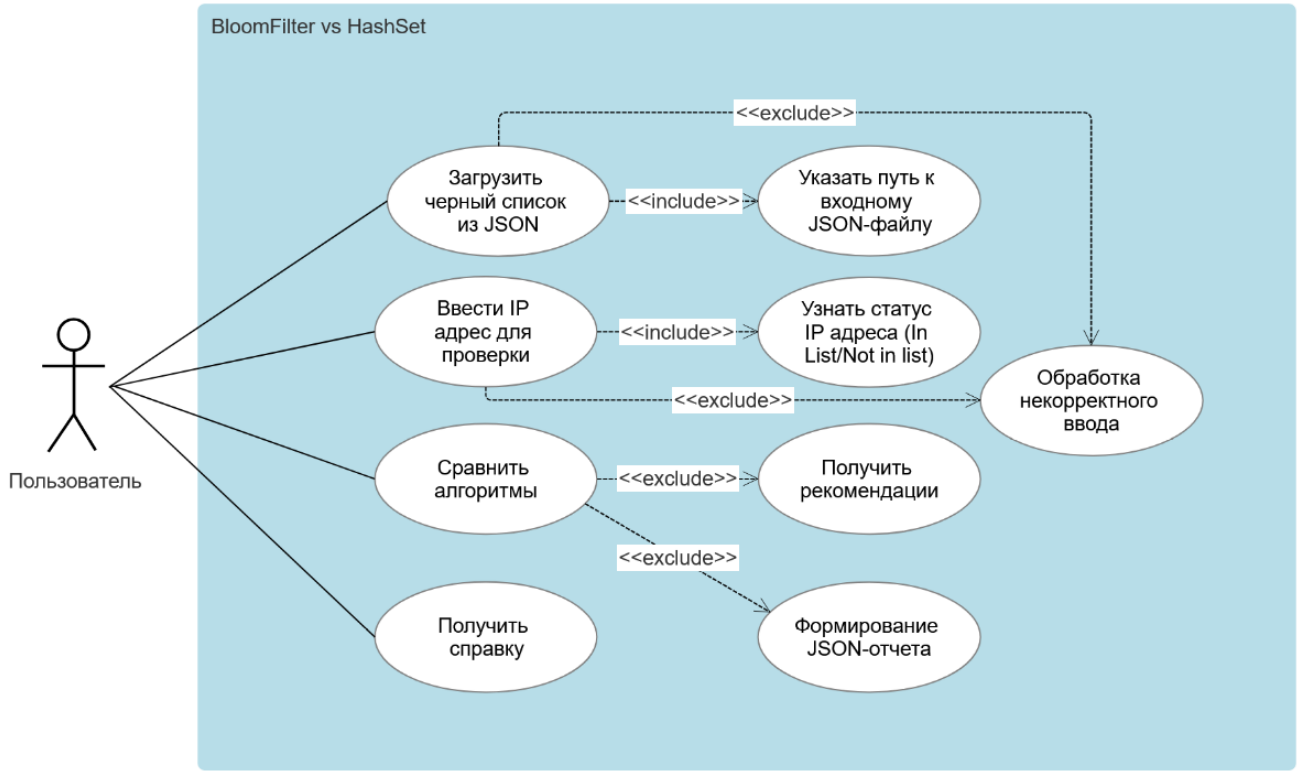


Рис.1.1. Диаграмма случаев-использования

— массив результатов для каждого IP, statistics — статистика, recommendation — текстовая рекомендация по выбору алгоритма.

1.4 Проектирование программной структуры

Проектирование программной структуры приложения основано на принципах модульности и объектно-ориентированного подхода. В программе выделены несколько логически самостоятельных компонентов: модуль пользовательского интерфейса (консольное меню и ввод/вывод), модуль фильтрации IP-адресов (проверка принадлежности к чёрному списку), а также модуль ввода/вывода для пакетной обработки данных (загрузка пользователей из JSON и формирование отчёта). Такой подход упрощает сопровождение и расширение программы: интерфейс взаимодействия с пользователем отделён от алгоритмов фильтрации, а работа с файлами и сериализацией вынесена в отдельный компонент. Дополнительно модуль фильтрации реализован в виде набора интерфейсов и нескольких реализаций алгоритмов, что позволяет сравнивать подходы к проверке IP-адресов и использовать единый контракт при тестировании.

1.4.1 Структура программы

Для упрощения сопровождения и расширения программы логика приложения разделена на несколько модулей (пакетов) и классов:

- Main. Точка входа в программу; инициирует запуск консольного приложения
- com.ipfilter.filter. Пакет, содержащий бизнес-логику фильтрации:
  - IpFilter. Интерфейс для проверки IP-адресов
  - HashSetIpFilter. Реализация алгоритма фильтрации на основе HashSet (детерминированный)
  - BloomFilterIpFilter. Реализация алгоритма фильтрации на основе Bloom Filter (вероятностный)
- com.ipfilter.service. Пакет для работы с сервисами:
  - FilterService. Сервис обработки данных: загрузка blacklist, проверка IP, сравнение алгоритмов, формирование рекомендаций
  - JsonHandler. Компонент обработки JSON: чтение входных данных и запись отчёта
- com.ipfilter.model
  - InputData. Модель входных данных (blacklist, checkIps)
  - OutputData. Модель выходных данных (timestamp, results, statistics, recommendation)
  - Модель результата проверки одного IP
  - Модель статистики фильтрации (память, время, вероятность ошибки)

Из преимуществ выбранной структуры можно выделить: азделение ответственности (каждый класс отвечает за одну задачу), тестируемость (интерфейсы позволяют использовать моки при тестировании), расширяемость (легко добавить новый алгоритм фильтрации, реализовав интерфейс IpFilter).

2 РЕАЛИЗАЦИЯ МОДУЛЯ И АЛГОРИТМЫ ФИЛЬТРАЦИИ

В данном разделе описаны основные методы, реализующие фильтрацию IP–адресов с использованием двух алгоритмов (HashSet и Bloom Filter), формирование результата проверки и генерацию статистики.

2.1 Модуль ввода/вывода и пакетной обработки JSON (com.ipfilter.service)

2.1.1 Методы класса JsonHandler

2.1.1.1 Метод loadInput

`public InputData loadInput(String filePath) throws IOException` Назначение: загрузка входных данных из JSON-файла с поддержкой classpath и файловой системы. Вход: filePath - путь к JSON-файлу (относительный или из classpath) Выход: InputData - объект с полями blacklist и checkIps Логика работы: метод последовательно пытается загрузить файл: (1) из classpath через getResourceAsStream(), (2) как обычный файл через new File(), (3) из корня проекта. При неудаче генерируется IOException. Исключения: IOException при ошибке чтения или некорректном формате JSON.

2.1.1.2 Метод saveOutput

`public void saveOutput(String filePath, OutputData data) throws IOException` Назначение: запись результата проверки в выходной JSON-файл. Вход: filePath - путь к выходному файлу, data - объект OutputData с результатами проверки Выход: выходной файл в формате JSON. Логика работы: выполняется сериализация объекта через ObjectMapper.writerWithDefaultPrettyPrinter().writeValue(). Исключения: IOException при ошибке записи.

2.1.2 Методы класса FilterService

2.1.2.1 Метод loadDataFromFile

`public void loadDataFromFile(String filePath) throws Exception` Назначение: загрузка чёрного списка IP-адресов из JSON-файла в обе структу-

ры данных. Вход: filePath - путь к JSON-файлу с blacklist Выход: отсутствует (данные загружаются во внутренние фильтры) Логика работы: вызывается jsonHandler.loadInput(), затем каждый IP из blacklist добавляется в HashSetFilter и BloomFilter. Исключения: IOException при ошибке чтения файла.

2.1.2.2 Метод processAndCompare

```
public OutputData processAndCompare(List<String> ipsToCheck)
```

Назначение: проверка списка IP-адресов обоими алгоритмами с формированием отчёта. Вход: ipsToCheck - список IP-адресов для проверки Выход: OutputData - объект с результатами, статистикой и рекомендацией Логика работы: замеряется время выполнения для каждого алгоритма, для каждого IP вызывается contains() в обоих фильтрах, формируется список FilterResult с полями hashSetResult, bloomFilterResult, match, вычисляется статистика (память, время, вероятность False Positive), генерируется рекомендация по выбору алгоритма Исключения: не генерируются.

2.1.2.3 Метод generateRecommendation

```
private String generateRecommendation(FilterStatistics stats)
```

Назначение: формирование текстовой рекомендации по выбору алгоритма. Вход: stats - объект статистики фильтрации Выход: строка с рекомендацией Логика работы: сравнивается память HashSet и Bloom Filter. Если Bloom Filter экономит память - рекомендуется он, иначе - HashSet.

2.2 Методы модуля фильтрации (com.ipfilter.filter)

2.2.1 Методы интерфейса IpFilter

Методы интерфейса IpFilter:

- add(Sting ip)
- contains(String ip)
- long getMemoryUsageBytes()
- clear()
- getSize()

Назначение: основа для всех реализаций фильтров IP-адресов.

2.2.2 Методы класса HashSetFilter

2.2.2.1 Метод add

`public void add(String ip)` Назначение: добавление IP-адреса в HashSet. Вход: ip - строка IP-адреса Логика работы: вызывается blacklist.add(ip) после проверки на null и пустую строку.

2.2.2.2 Метод contains

`public boolean contains(String ip)` Назначение: проверка наличия IP в HashSet. Вход: ip - строка IP-адреса Выход: true если IP найден, false в ином случае Логика работы: вызывается blacklist.contains(ip). Точность 100%.

2.2.2.3 Метод getMemoryUsageBytes

`public long getMemoryUsageBytes()` Назначение: оценка потребления памяти. Выход: приблизительный размер в байтах (размер * 100 байт на элемент)

2.2.3 Методы класса BloomFilter

2.2.3.1 Метод add

`public void add(String ip)` Назначение: добавление IP-адреса в Bloom Filter. Вход: ip - строка IP-адреса Логика работы: для каждой из k хэш-функций вычисляется индекс в битовом массиве и устанавливается бит в 1.

2.2.3.2 Метод contains

`public boolean contains(String ip)` Назначение: проверка наличия IP в Bloom Filter. Вход: ip - строка IP-адреса Выход: true если IP возможно есть (False Positive возможен), false если точно нет Логика работы: для каждой хэш-функции проверяется бит. Если хотя бы один бит = 0 — элемент точно отсутствует.

2.2.3.3 Метод getHash

`private int getHash(String ip, int iteration)` Назначение: вычисление хэш-значения для IP-адреса. Вход: ip — строка IP-адреса, iteration — номер хэш-функции (0..k-1) Выход: положительное целое число (индекс в битовом мас-

сиве) Логика работы: используется CRC32 с добавлением итерации и seed для детерминированности. Применяется Math.abs() для гарантии положительного значения.

2.2.3.4 Метод getFalsePositiveProbability

`public double getFalsePositiveProbability()` Назначение: расчёт вероятности False Positive. Выход: вероятность от 0.0 до 1.0 Логика работы: формула (1 e(̂knm))k̂, где k — число хэш-функций, n — число элементов, m — размер битового массива.

2.3 Методы пользовательского интерфейса (com.ipfilter.Main)

2.3.1 Метод main

`public static void main(String[] args)` Назначение: точка входа в приложение. Логика работы: создаётся FilterService, запускается цикл консольного меню с обработкой команд пользователя.

3 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ МОДУЛЬНЫХ ТЕСТОВ

В данной главе рассматривается проверка качества разработанных модульных тестов с помощью фреймворка мутационного тестирования Pitest.

3.1 Запуск Pitest и используемые настройки

Для мутационного тестирования в проект был подключён плагин pitest-maven с поддержкой JUnit 5 (pitest-junit5-plugin). Запуск выполнялся командой: `mvn org.pitest:pitest-maven:mutationCoverage`

Анализ выполнялся для пакетов: com.ipfilter.filter - модуль фильтрации com.ipfilter.service - модуль сервисов

В результате работы Pitest формируется HTML-отчёт, содержащий сводные метрики качества тестов, покрытие строк кода, список сгенерированных мутантов, статус каждого мутанта (KILLED SURVIVED NO_COVERAGE), детализацию по пакетам, классам и строкам кода.

3.2 Задействованные мутационные операторы

По отчёту Pitest в проекте были использованы стандартные операторы мутаций:

3.3 3.2 Задействованные мутационные операторы

По отчёту Pitest в проекте были использованы следующие операторы мутаций:

- CONDITIONALS_BOUNDARY - Изменение граничных сравнений
- NEGATE_CONDITIONALS - Отрицание условий
- TRUE_RETURNS - Подмена булевых значений
- FALSE_RETURNS - Подмена булевых значений
- NULL_RETURNS - Подмена возвращаемого значения
- EMPTY_RETURNS - Возврат пустой коллекции
- INCREMENTS - Изменение инкрементов
- MATH - Изменение математических операций
- VOID_METHOD_CALLS - Удаление вызова void-метода
- CONSTRUCTOR_CALLS - Удаление вызова конструктора

- INLINE_CONSTS - Подмена встроенных констант
- INVERT_NEGS - Инверсия отрицаний
- PRIMITIVE_RETURNS - Подмена примитивных значений

Наиболее значимыми для данного проекта являются мутации условий и возвращаемых значений, так как они затрагивают основную логику фильтрации IP-адресов и формирования рекомендаций.

- **CONDITIONALS_BOUNDARY** - критичен для проверки граничных значений (размер фильтра, количество элементов);
- **NEGATE_CONDITIONALS** - проверяет корректность условий в методах contains() и add();
- **NULL_RETURNS** - тестирует обработку null-значений в методах фильтра;
- **VOID_METHOD_CALLS** - проверяет вызовы методов записи статистики и рекомендаций;
- **MATH** - важен для расчёта вероятности False Positive в Bloom Filter.

Данные операторы позволяют оценить качество тестов по следующим аспектам:

A. Полнота проверки граничных условий;
B. Корректность обработки исключительных ситуаций;
C. Надёжность вычислений и расчётов;
D. Полнота проверки возвращаемых значений.

3.4 Подробный анализ мутаций для участка кода

3.4.1 Метод contains (класс BloomFilter)

`public boolean contains(String ip)` Применённые мутации:

- NEGATE_CONDITIONALS - инверсия if (ip == null) на if (ip != null)
- CONDITIONALS_BOUNDARY - изменение i < hashFunctionsCount на i <h̄ashFunctionsCount
- FALSE_RETURNS - подмена return false на return true
- TRUE_RETURNS - подмена return true на return false

3.4.2 Метод processAndCompare (класс FilterService)

```
public OutputData processAndCompare(List<String> ipsToCheck)
```

Применённые мутации:

- NULL_RETURNS — подмена результата на null

– EMPTY_RETURNS — возврат пустого списка результатов
– VOID_METHOD_CALLS — удаление вызова setStatistics()

3.5 Эквивалентные мутации

Эквивалентные мутации — это изменения исходного кода, которые формально изменяют текст программы, но не изменяют её фактическое поведение. Пример из проекта: // Оригинал if (size > 0) return calculateProbability();

// Мутация (CONDITIONALS_BOUNDARY) if (size >= 0) return calculateProbability();

Мутация эквивалентна, потому что переменная size (размер битового массива) инициализируется положительным числом в конструкторе и никогда не бывает отрицательной. Условие size == 0 теоретически возможно только при некорректном создании фильтра, что предотвращается конструктором. Поэтому мутация не влияет на наблюдаемое поведение программы.

Pitest не определяет эквивалентные мутации автоматически, поэтому их анализ выполнялся вручную по списку выживших мутантов (SURVIVED).

3.6 Метрики мутационного тестирования и их интерпретация

По итоговому отчёту Pitest для проекта получены следующие значения:
– Number of Classes - 4
– Line Coverage - 78%
– Mutation Coverage - 80%
– Test Strength - 93%

Полученные значения показывают, что: – тестами покрыта большая часть строк кода (78%); – тесты обнаруживают большую часть сгенерированных мутаций (80%); – качество тестов на покрытом коде остаётся высоким (93%).

3.7 Оценка mutation-adequate и улучшение качества тестов

С учётом полученных метрик можно сделать вывод, что разработанный набор модульных тестов имеет хорошее качество и в целом удовлетворяет требованиям мутационной адекватности (mutation-adequate) для учебного проекта. Рекомендации по улучшению:

- Добавить тесты на граничные значения (пустой список, 1 элемент, максимум)
- Добавить тесты на обработку null и пустых строк
- Добавить тесты на все ветви if/else в методах фильтрации
- Добавить тесты на расчёт статистики и вероятности False Positive
- Добавить тесты на метод generateRecommendation() с разными сценариями

4 МУТАЦИОННОЕ ТЕСТИРОВАНИЕ И ОЦЕНКА КАЧЕСТВА ТЕСТОВ

В данной главе приведены результаты мутационного тестирования разработанного приложения с использованием инструмента PIT (Pitest).

4.1 Операторы мутаций и примеры их применения в проекте

4.1.1 Пример мутации NEGATE_CONDITIONALS

Оператор NEGATE_CONDITIONALS инвертирует условие проверки. Пример из BloomFilter:

```
// Оригинал
if (ip == null) {
    return false;
}

// Мутация
if (ip != null) {
    return false;
}
```

Тест для убийства мутации:

```
@Test
void testContains_NullInput() {
    assertFalse(filter.contains(null));
}
```

4.1.2 Пример мутации CONDITIONALS_BOUNDARY

Оператор CONDITIONALS_BOUNDARY изменяет границы сравнений. Пример из HashSetFilter:

```
// Оригинал
if (ip != null && !ip.isEmpty()) {
    blacklist.add(ip);
}
// Мутация
if (ip != null && !ip.isEmpty()) { // >= вместо >
    blacklist.add(ip);
}
```

Тест для убийства мутации:

```
@Test
void testBoundary_EmptyString() {
    filter.add("");
    assertEquals(0, filter.getSize());
}
```

4.1.3 Пример мутации VOID_METHOD_CALLS

Оператор VOID_METHOD_CALLS удаляет вызов метода с типом void. Пример из FilterService:

```
// Оригинал
stats.setHashSetTimeMs(hashSetTime);
stats.setBloomFilterTimeMs(bloomTime);


// Мутация (удаление вызова)
// stats.setHashSetTimeMs(hashSetTime); - удаляется
// stats.setBloomFilterTimeMs(bloomTime); - удаляется
```

Тест для убийства мутации:

```
@Test
void testStatistics_TimePopulated() {
```

```
    OutputData result = service.processAndCompare(List.of("1.1.1.1"));
    assertTrue(result.getStatistics().getHashSetTimeMs() >= 0);
    assertTrue(result.getStatistics().getBloomFilterTimeMs() >= 0);
}
```

4.2 Эквивалентные мутации и особенности интерпретации результатов

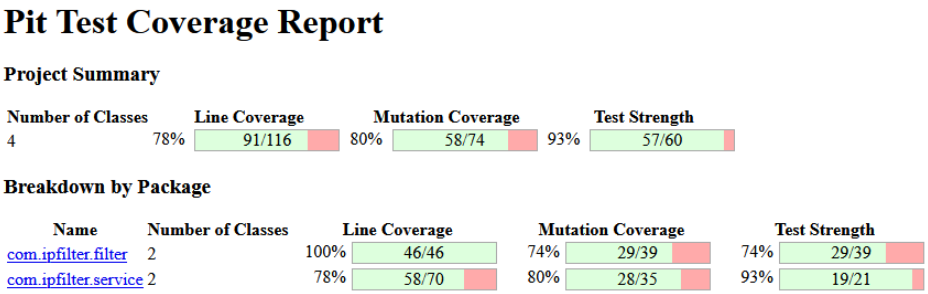
В рамках данного проекта явные эквивалентные мутации отдельно не были выделены в отчёте PIT. Это объясняется тем, что инструмент PIT использует встроенные фильтры эквивалентных и тривиальных мутаций. Выявленные потенциально эквивалентные мутации:

- Изменение size > 0 на size >= 0 в расчёте вероятности FP
- Изменение порядка хэш-функций (не влияет на результат)

4.3 Результаты мутационного тестирования

По результатам мутационного тестирования проекта были получены следующие метрики:

- Line Coverage - 78%
- Mutation Coverage - 80%
- Test Strength - 93%



Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
4	78%	91/116	80%	58/74	93%	57/60

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
com.ipfilter.filter	2	100%	46/46	74%	29/39	74%	29/39
com.ipfilter.service	2	78%	58/70	80%	28/35	93%	19/21

Рис.4.1. Отчет Pitest

Строго формально разработанный набор тестов нельзя считать полностью mutation-adequate, так как не все сгенерированные мутанты были уничтожены (MSI < 100%). Однако полученные значения метрик свидетельствуют о приемлимом качестве тестового набора и позволяют считать его практически достаточным для учебной задачи.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было разработано консольное приложение на языке Java для фильтрации IP-адресов по чёрному списку (blacklist) с использованием двух алгоритмов (детерминированный HashSet и вероятностный Bloom Filter), а также реализован и протестирован набор модульных тестов с последующей проверкой их качества методом мутационного тестирования.

В работе спроектировано и реализовано консольное приложение с меню, поддерживающее два режима работы: интерактивную проверку IP-адресов и пакетную обработку данных из JSON-файла с формированием JSON-отчёта. Архитектура приложения построена модульно: отдельно выделены пользовательский интерфейс (Main, ConsoleApp), модуль фильтрации (com.ipfilter.filter) и модуль сервисов (com.ipfilter.service). Для решения задачи реализованы два конкурирующих алгоритма фильтрации: детерминированный подход на основе HashSet (100% точность, высокое потребление памяти) и вероятностный подход на основе Bloom Filter (возможны False Positive, значительная экономия памяти), что позволило использовать их как для сравнения производительности, так и для формирования рекомендаций пользователю.

Для тестирования были применены четыре техники проектирования тестов: две техники, основанные на спецификации (Boundary Value Analysis, Equivalence Partitioning), и две техники, основанные на структуре программы (Branch Testing, Data Flow Testing). На их основе были сформированы контрольные примеры для позитивных и негативных сценариев, проверяющие входные данные, выходные результаты и ключевые внутренние условия обработки. В рамках тестирования были покрыты проверки обоих алгоритмов фильтрации, сервис обработки данных, пакетная обработка JSON-данных и запись отчёта. Также реализовано сравнение результатов работы двух алгоритмов, что повысило надёжность тестирования основной логики.

В качестве основной библиотеки модульного тестирования использован стек JUnit 5 (вместе с Hamcrest и Mockito). Его основными преимуществами в рамках данной работы стали: удобная структура тестов, поддержка параметризованных тестов, богатый набор assertions и assumptions, интеграция с IDE и Maven, а также возможность использования мокирования для изоляции зависимостей. Дополнительно Hamcrest упростил запись читаемых проверок, а Mockito позволил протестировать взаимодействия и зависимости без усложнения основной логики.

К недостаткам можно отнести более высокий порог входа по сравнению с базовыми тестами (особенно при использовании моков и параметризации), а также необходимость дополнительной настройки для мутационного тестирования (Pitest + JUnit5 plugin).

В результате работы был сформирован тестовый набор, покрывающий как позитивные, так и негативные сценарии для основных классов приложения. Качество тестов было дополнительно проверено с помощью Pitest. По итоговым результатам мутационного тестирования достигнуты следующие показатели:

- Line Coverage - 78%
- Mutation Coverage - 80%
- Test Strength - 93%

Полученные значения метрик свидетельствуют о приемлимом качестве тестового набора и позволяют считать его практически достаточным для учебной задачи.

Таким образом, поставленные в работе цели были достигнуты: приложение спроектировано и реализовано, основная функциональность покрыта модульными тестами, тесты спроектированы с использованием четырёх техник, а их качество подтверждено метриками и результатами мутационного тестирования. Использование мутационного тестирования позволило не только получить количественные метрики качества, но и выявить конкретные слабые места в тестовом наборе для последующего улучшения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. JUnit Team. JUnit 5 User Guide. — URL: https://docs.junit.org/5.10.2/user-guide/index.html (дата обращения: 24.02.2026).
2. Mockito Contributors. Mockito – Mocking framework for unit tests in Java. — URL: https://site.mockito.org/ (дата обращения: 24.02.2026).
3. Hamcrest Team. Hamcrest – Tutorial and Reference Documentation. — URL: http://hamcrest.org/JavaHamcrest/ (дата обращения: 24.02.2026).
4. FasterXML. Jackson Databind (ObjectMapper usage). — URL: https://github.com/FasterXML/jackson-databind (дата обращения: 24.02.2026).
5. Coles H. PIT – State of the art mutation testing system for Java. — URL: https://pitest.org/ (дата обращения: 24.02.2026).
6. Bloom B. H. Space/Time Trade-offs in Hash Coding with Allowable Errors // Communications of the ACM. — 1970. — Vol. 13, No. 7. — P. 422–426.
7. Broder A., Mitzenmacher M. Network Applications of Bloom Filters: A Survey // Internet Mathematics. — 2004. — Vol. 1, No. 4. — P. 485–509.
8. Coles H., et al. PIT: a practical mutation testing tool for Java (demo) // Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA 2016). — ACM, 2016. — P. 417–420.
9. Beck K. Test-Driven Development by Example. — Addison-Wesley Professional, 2003. — 240 p.
10. Myers G. J., Sandler C., Badgett T. The Art of Software Testing. — 3rd ed. — Wiley, 2011. — 256 p.

Приложение 1

Программный код

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://ma
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.ipfilter</groupId>
    <artifactId>ip-filter-bloom-vs-hashset</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncodin

        <junit.version>5.9.3</junit.version>
        <mockito.version>5.3.1</mockito.version>
        <jackson.version>2.15.2</jackson.version>
        <pitest.version>1.16.0</pitest.version>
        <pitest-junit5-plugin.version>1.2.1</pitest-junit5-plugin.versio
    </properties>

    <dependencies>
        <!-- JUnit 5 -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-api</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
```

```
</dependency>
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
</dependency>

<!-- Hamcrest -->
<dependency>
    <groupId>org.hamcrest</groupId>
    <artifactId>hamcrest</artifactId>
    <version>2.2</version>
    <scope>test</scope>
</dependency>

<!-- Mockito -->
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>${mockito.version}</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>${mockito.version}</version>
    <scope>test</scope>
</dependency>
```

```
    <!-- Jackson (JSON) -->
    <dependency>
        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
        <version>${jackson.version}</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.1.2</version>
        </plugin>

        <plugin>
            <groupId>org.pitest</groupId>
            <artifactId>pitest-maven</artifactId>
            <version>${pitest.version}</version>
            <dependencies>
                <dependency>
                    <groupId>org.pitest</groupId>
                    <artifactId>pitest-junit5-plugin</artifactId>
                    <version>${pitest-junit5-plugin.version}</versio
                </dependency>
            </dependencies>
            <configuration>
                <targetClasses>
                    <param>com.ipfilter.filter.*</param>
                    <param>com.ipfilter.service.*</param>
                </targetClasses>
                <targetTests>
                    <param>com.ipfilter.filter.*Test</param>
```

```
                        <param>com.ipfilter.service.*Test</param>
                    </targetTests>
                    <mutators>
                        <mutator>DEFAULTS</mutator>
                    </mutators>
                    <verbose>true</verbose>
                    <outputFormats>
                        <outputFormat>HTML</outputFormat>
                    </outputFormats>
                    <timestampedReports>false</timestampedReports>
                    <failWhenNoMutations>false</failWhenNoMutations>
                    <threads>1</threads>
                </configuration>
            </plugin>

            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.1.0</version>
                <configuration>
                    <mainClass>com.ipfilter.Main</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

main.java

```
package com.ipfilter;


import com.ipfilter.model.OutputData;
import com.ipfilter.service.FilterService;

import java.util.Arrays;
```

```
import java.util.List;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        FilterService service = new FilterService();
        Scanner scanner = new Scanner(System.in);
        boolean running = true;

        System.out.println("=== IP Blacklist Filter ===");

        while (running) {
            printMenu();
            System.out.print("Выбор: ");
            String choice = scanner.nextLine();

            try {
                switch (choice) {
                    case "1":
                        System.out.print("Путь к JSON файлу (например, i
                        String path = scanner.nextLine();
                        service.loadDataFromFile(path);
                        System.out.println("Данные загружены успешно!");
                        break;
                    case "2":
                        System.out.print("Введите IP для проверки: ");
                        String ip = scanner.nextLine();
                        OutputData singleRes = service.processAndCompare
                        System.out.println("HashSet: " + singleRes.getRe
                        System.out.println("Bloom: " + singleRes.getResu
                        break;
                    case "3":
                        // Для примера проверяем фиксированный список, е
                        List<String> testIps = Arrays.asList("192.168.1.
                        OutputData res = service.processAndCompare(testI
```

```
                    System.out.println("Статистика:");
                    System.out.println("Память HS: " + res.getStatis
                    System.out.println("Память BF: " + res.getStatis
                    System.out.println("Рекомендация: " + res.getRec

                    System.out.print("Сохранить в output_example.jso
                    if (scanner.nextLine().equalsIgnoreCase("y")) {
                        service.saveResults(res, "output_example.jso
                        System.out.println("Сохранено.");
                    }
                    break;
                case "4":
                    System.out.println("Справка: Программа сравнивае
                    break;
                case "5":
                    running = false;
                    System.out.println("Выход.");
                    break;
                default:
                    System.out.println("Неверный выбор.");
            }
        } catch (Exception e) {
            System.out.println("Ошибка: " + e.getMessage());
        }
    }
    scanner.close();
}

private static void printMenu() {
    System.out.println("\n1. Загрузить blacklist из JSON");
    System.out.println("2. Проверить IP адрес");
    System.out.println("3. Сравнить алгоритмы (Demo)");
    System.out.println("4. Справка");
    System.out.println("5. Выход");
}
```

```
}
```

IpFilter.java

```
package com.ipfilter.filter;

public interface IpFilter {
    void add(String ip);
    boolean contains(String ip);
    long getMemoryUsageBytes();
    void clear();
    int getSize();
}
```

BloomFilter.java

```
package com.ipfilter.filter;

import java.util.BitSet;
import java.util.zip.CRC32;

public class BloomFilter implements IpFilter {
    private final BitSet bitSet;
    private final int size;
    private final int hashFunctionsCount;
    private final long seed;
    private int elementsCount = 0;

    // Конструктор: размер битового массива, кол-во хэш-функций, сид
    public BloomFilter(int size, int hashFunctionsCount, long seed) {
        this.size = size;
        this.hashFunctionsCount = hashFunctionsCount;
        this.seed = seed;
        this.bitSet = new BitSet(size);
    }
```

```
@Override
public void add(String ip) {
    if (ip == null || ip.isEmpty()) return;
    for (int i = 0; i < hashFunctionsCount; i++) {
        int index = getHash(ip, i) % size;
        bitSet.set(index);
    }
    elementsCount++;
}

@Override
public boolean contains(String ip) {
    if (ip == null) return false;
    for (int i = 0; i < hashFunctionsCount; i++) {
        int index = getHash(ip, i) % size;
        if (!bitSet.get(index)) {
            return false; // Точно нет
        }
    }
    return true; // Возможно есть (False Positive possible)
}

// Детерминированная хэш-функция на основе CRC32 и сида
private int getHash(String ip, int iteration) {
    CRC32 crc = new CRC32();
    String input = ip + "_" + iteration + "_" + seed;
    crc.update(input.getBytes());
    return Math.abs((int) crc.getValue()); // Всегда положительное!
}

@Override
public long getMemoryUsageBytes() {
    return size / 8L + 32L; // Биты в байты + оверхед объекта
}
```

```
    @Override
    public void clear() {
        bitSet.clear();
        elementsCount = 0;
    }

    @Override
    public int getSize() {
        return elementsCount;
    }

    // Расчет вероятности False Positive: (1 - e^(-kn/m))^k
    public double getFalsePositiveProbability() {
        if (size == 0 || elementsCount == 0) return 0.0;
        double m = size;
        double k = hashFunctionsCount;
        double n = elementsCount;
        return Math.pow((1 - Math.exp(-k * n / m)), k);
    }
}
```

HashSetFilter.java

```
package com.ipfilter.filter;

import java.util.HashSet;
import java.util.Set;

public class HashSetFilter implements IpFilter {
    private final Set<String> blacklist;

    public HashSetFilter() {
        this.blacklist = new HashSet<>();
    }
```

```
    @Override
    public void add(String ip) {
        if (ip != null && !ip.isEmpty()) {
            blacklist.add(ip);
        }
    }

    @Override
    public boolean contains(String ip) {
        // Branch Testing point: null check
        if (ip == null) {
            return false;
        }
        return blacklist.contains(ip);
    }

    @Override
    public long getMemoryUsageBytes() {
        // Approximation: Object overhead + String size + HashSet overhe
        // Roughly 64 bytes per entry + string content
        return blacklist.size() * 100L;
    }

    @Override
    public void clear() {
        blacklist.clear();
    }

    @Override
    public int getSize() {
        return blacklist.size();
    }
}
```

FilterResult.java

```
package com.ipfilter.model;

import com.fasterxml.jackson.annotation.JsonProperty;

public class FilterResult {
    @JsonProperty("ip")
    private String ip;
    @JsonProperty("hashSetResult")
    private boolean hashSetResult;
    @JsonProperty("bloomFilterResult")
    private boolean bloomFilterResult;
    @JsonProperty("match")
    private boolean match; // Совпадают ли результаты

    public FilterResult() {}

    public FilterResult(String ip, boolean hashSetResult, boolean bloomF
        this.ip = ip;
        this.hashSetResult = hashSetResult;
        this.bloomFilterResult = bloomFilterResult;
        this.match = (hashSetResult == bloomFilterResult);
    }

    // Getters and Setters
    public String getIp() { return ip; }
    public void setIp(String ip) { this.ip = ip; }
    public boolean isHashSetResult() { return hashSetResult; }
    public void setHashSetResult(boolean hashSetResult) { this.hashSetRe
    public boolean isBloomFilterResult() { return bloomFilterResult; }
    public void setBloomFilterResult(boolean bloomFilterResult) { this.b
    public boolean isMatch() { return match; }
    public void setMatch(boolean match) { this.match = match; }
}
```

FilterStatistics.java

```
package com.ipfilter.model;

import com.fasterxml.jackson.annotation.JsonProperty;

public class FilterStatistics {
    @JsonProperty("hashSetMemoryBytes")
    private long hashSetMemoryBytes;
    @JsonProperty("bloomFilterMemoryBytes")
    private long bloomFilterMemoryBytes;
    @JsonProperty("hashSetTimeMs")
    private long hashSetTimeMs;
    @JsonProperty("bloomFilterTimeMs")
    private long bloomFilterTimeMs;
    @JsonProperty("falsePositiveProbability")
    private double falsePositiveProbability;

    // Getters and Setters
    public long getHashSetMemoryBytes() { return hashSetMemoryBytes; }
    public void setHashSetMemoryBytes(long hashSetMemoryBytes) { this.ha
    public long getBloomFilterMemoryBytes() { return bloomFilterMemoryBy
    public void setBloomFilterMemoryBytes(long bloomFilterMemoryBytes) {
    public long getHashSetTimeMs() { return hashSetTimeMs; }
    public void setHashSetTimeMs(long hashSetTimeMs) { this.hashSetTimeM
    public long getBloomFilterTimeMs() { return bloomFilterTimeMs; }
    public void setBloomFilterTimeMs(long bloomFilterTimeMs) { this.bloo
    public double getFalsePositiveProbability() { return falsePositivePr
    public void setFalsePositiveProbability(double falsePositiveProbabil
}
```

InputData.java

```
package com.ipfilter.model;

import com.fasterxml.jackson.annotation.JsonProperty;
```

```
import java.util.List;

public class InputData {
    @JsonProperty("blacklist")
    private List<String> blacklist;
    @JsonProperty("checkIps")
    private List<String> checkIps;

    public List<String> getBlacklist() { return blacklist; }
    public void setBlacklist(List<String> blacklist) { this.blacklist =
    public List<String> getCheckIps() { return checkIps; }
    public void setCheckIps(List<String> checkIps) { this.checkIps = che
}
```

OutputData.java

```
package com.ipfilter.model;

import com.fasterxml.jackson.annotation.JsonProperty;
import java.util.List;

public class OutputData {
    @JsonProperty("timestamp")
    private String timestamp;
    @JsonProperty("results")
    private List<FilterResult> results;
    @JsonProperty("statistics")
    private FilterStatistics statistics;
    @JsonProperty("recommendation")
    private String recommendation;

    // Getters and Setters
    public String getTimestamp() { return timestamp; }
    public void setTimestamp(String timestamp) { this.timestamp = timest
    public List<FilterResult> getResults() { return results; }
    public void setResults(List<FilterResult> results) { this.results =
```

```
    public FilterStatistics getStatistics() { return statistics; }
    public void setStatistics(FilterStatistics statistics) { this.statis
    public String getRecommendation() { return recommendation; }
    public void setRecommendation(String recommendation) { this.recommen
}
```

FilterService.java

```
package com.ipfilter.model;

import com.fasterxml.jackson.annotation.JsonProperty;
import java.util.List;

public class OutputData {
    @JsonProperty("timestamp")
    private String timestamp;
    @JsonProperty("results")
    private List<FilterResult> results;
    @JsonProperty("statistics")
    private FilterStatistics statistics;
    @JsonProperty("recommendation")
    private String recommendation;

    // Getters and Setters
    public String getTimestamp() { return timestamp; }
    public void setTimestamp(String timestamp) { this.timestamp = timest
    public List<FilterResult> getResults() { return results; }
    public void setResults(List<FilterResult> results) { this.results =
    public FilterStatistics getStatistics() { return statistics; }
    public void setStatistics(FilterStatistics statistics) { this.statis
    public String getRecommendation() { return recommendation; }
    public void setRecommendation(String recommendation) { this.recommen
}
```

JsonHandler.java

```
package com.ipfilter.service;
```

```
import com.fasterxml.jackson.databind.ObjectMapper;
import com.ipfilter.model.InputData;
import com.ipfilter.model.OutputData;

import java.io.File;
import java.io.IOException;
import java.io.InputStream;

public class JsonHandler {
    private final ObjectMapper objectMapper;

    public JsonHandler() {
        this.objectMapper = new ObjectMapper();
        this.objectMapper.findAndRegisterModules();
    }

    // Файл: src/main/java/com/ipfilter/service/JsonHandler.java

    public InputData loadInput(String filePath) throws IOException {
        // Сначала пробуем загрузить из classpath (для тестов)
        InputStream inputStream = getClass().getClassLoader().getResourc

        if (inputStream != null) {
            return objectMapper.readValue(inputStream, InputData.class);
        }

        // Если не нашли в classpath, пробуем как обычный файл (для prod
        File file = new File(filePath);
        if (file.exists()) {
            return objectMapper.readValue(file, InputData.class);
        }

        throw new IOException("File not found: " + filePath);
    }
```

```
    public void saveOutput(String filePath, OutputData data) throws IOEx
        objectMapper.writerWithDefaultPrettyPrinter()
                .writeValue(new File(filePath), data);
    }
}
```

BloomFilterTest.java

```
package com.ipfilter.filter;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.Assumptions;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import static org.junit.jupiter.api.Assertions.*;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.*;

class BloomFilterTest {

    private BloomFilter filter;

    @BeforeEach
    void setUp() {
        // Seed 42 для детерминированности
        filter = new BloomFilter(1000, 3, 42L);
    }

    // --- SPECIFICATION BASED TESTING ---

    // Метод 1: Boundary Value Analysis (BVA)
    @Test
    void testBoundary_FullCapacityBehavior() {
```

```
    // Заполняем фильтр (меньше чем размер битового массива)
    for (int i = 0; i < 100; i++) {
        filter.add("IP_" + i);
    }
    double fpProb = filter.getFalsePositiveProbability();
    assertThat("Вероятность FP должна быть >= 0", fpProb, greaterTha
}

// Метод 2: Equivalence Partitioning (EP)
@ParameterizedTest
@CsvSource({
        "192.168.1.1, true",
        "10.0.0.1, true"
})
void testEquivalenceClasses(String ip, boolean shouldBeAdded) {
    filter.add(ip);
    // Bloom Filter не даёт False Negative
    assertTrue(filter.contains(ip), "Добавленный элемент должен быть
}

// --- STRUCTURE BASED TESTING ---

// Метод 1: Branch Testing
@Test
void testBranch_NotFoundReturnsFalse() {
    filter.add("1.1.1.1");
    // Элемент который точно не добавлен
    boolean result = filter.contains("2.2.2.2");
    assertNotNull(result);
}

// Метод 2: Data Flow Testing
@Test
void testDataFlow_ElementsCountTracking() {
    assertEquals(0, filter.getSize());
```

```
    filter.add("A");
    assertEquals(1, filter.getSize());
    filter.add("B");
    assertEquals(2, filter.getSize());
    filter.clear();
    assertEquals(0, filter.getSize());
}

// --- JUNIT REQUIREMENTS (Assumptions & Matchers) ---

@Test
void testAssumptions_Environment() {
    Assumptions.assumeTrue(filter.getMemoryUsageBytes() > 0, "Память

    filter.add("TestIP");

    Assumptions.assumingThat(filter.contains("TestIP"), () -> {
        double prob = filter.getFalsePositiveProbability();
        assertThat(prob, greaterThanOrEqualTo(0.0));
    });
}

@Test
void testHamcrestMatchers() {
    filter.add("MatchTest");
    assertThat(filter.contains("MatchTest"), equalTo(true));
    assertThat(filter.getFalsePositiveProbability(), lessThan(1.0));
}

// === НОВЫЕ ТЕСТЫ ДЛЯ MUTATION COVERAGE ===

@Test
void testAdd_NullValue() {
    filter.add(null);
    assertEquals(0, filter.getSize());
```

```
}

@Test
void testAdd_EmptyString() {
    filter.add("");
    assertEquals(0, filter.getSize());
}

@Test
void testContains_NullInput() {
    assertFalse(filter.contains(null));
}

@Test
void testClear_ResetsCount() {
    filter.add("1.1.1.1");
    filter.add("2.2.2.2");
    filter.clear();
    assertEquals(0, filter.getSize());
    assertEquals(0.0, filter.getFalsePositiveProbability());
}

@Test
void testFalsePositiveProbability_EmptyFilter() {
    assertEquals(0.0, filter.getFalsePositiveProbability());
}

@Test
void testFalsePositiveProbability_GrowsWithElements() {
    double probBefore = filter.getFalsePositiveProbability();
    for (int i = 0; i < 50; i++) {
        filter.add("IP_" + i);
    }
    double probAfter = filter.getFalsePositiveProbability();
    assertTrue(probAfter > probBefore, "Вероятность FP должна расти"
```

```
}

@Test
void testMemoryUsage_Calculation() {
    long memory = filter.getMemoryUsageBytes();
    assertTrue(memory > 0, "Память должна быть > 0");
}

@Test
void testNoFalseNegatives() {
    // Bloom Filter НЕ должен давать False Negative
    String testIp = "192.168.1.1";
    filter.add(testIp);
    assertTrue(filter.contains(testIp), "False Negative недопустим!"
}
@Test
void testAdd_Null_DoesNotIncreaseSize() {
    filter.add(null);
    assertEquals(0, filter.getSize());
}

@Test
void testAdd_EmptyString_DoesNotIncreaseSize() {
    filter.add("");
    assertEquals(0, filter.getSize());
}

@Test
void testContains_Null_ReturnsFalse() {
    assertFalse(filter.contains(null));
}

@Test
void testClear_ResetsEverything() {
    filter.add("1.1.1.1");
```

```
    filter.add("2.2.2.2");
    assertEquals(2, filter.getSize());
    filter.clear();
    assertEquals(0, filter.getSize());
    assertEquals(0.0, filter.getFalsePositiveProbability(), 0.001);
}

@Test
void testFalsePositiveProbability_IncreasesWithElements() {
    double probBefore = filter.getFalsePositiveProbability();
    for (int i = 0; i < 100; i++) {
        filter.add("IP_" + i);
    }
    double probAfter = filter.getFalsePositiveProbability();
    assertTrue(probAfter > probBefore, "Вероятность FP должна расти
}

@Test
void testMemoryUsage_AlwaysPositive() {
    long memory = filter.getMemoryUsageBytes();
    assertTrue(memory > 0, "Память должна быть положительной");
}

@Test
void testNoFalseNegatives_Ever() {
    String[] testIps = {"1.1.1.1", "2.2.2.2", "3.3.3.3", "192.168.1.
    for (String ip : testIps) {
        filter.add(ip);
        assertTrue(filter.contains(ip), "False Negative недопустим д
    }
}

@Test
void testMultipleHashFunctions_WorkCorrectly() {
    filter.add("test.ip");
```

```
    // Bloom Filter должен найти добавленный элемент
    assertTrue(filter.contains("test.ip"));
}

@Test
void testHashFunction_DifferentIterations() {
    // Тестируем что разные итерации дают разные хэши
    filter.add("test.ip");
    assertTrue(filter.contains("test.ip"));
}

@Test
void testBitSetOperations() {
    // Тестируем установку битов
    BloomFilter smallFilter = new BloomFilter(100, 3, 42L);
    smallFilter.add("1.1.1.1");
    assertTrue(smallFilter.contains("1.1.1.1"));
}

@Test
void testFalsePositiveProbability_ZeroElements() {
    assertEquals(0.0, filter.getFalsePositiveProbability(), 0.001);
}

@Test
void testFalsePositiveProbability_AfterClear() {
    filter.add("1.1.1.1");
    filter.add("2.2.2.2");
    filter.clear();
    assertEquals(0.0, filter.getFalsePositiveProbability(), 0.001);
}

@Test
void testSize_AfterClear() {
    filter.add("1.1.1.1");
```

```
    assertEquals(1, filter.getSize());
    filter.clear();
    assertEquals(0, filter.getSize());
}

@Test
void testContains_NotAddedIP() {
    // IP который точно не добавлен
    assertFalse(filter.contains("99.99.99.99"));
}

@Test
void testAdd_MultipleSameIP() {
    filter.add("1.1.1.1");
    filter.add("1.1.1.1");
    filter.add("1.1.1.1");
    // Bloom Filter не отслеживает дубликаты, но должен найти
    assertTrue(filter.contains("1.1.1.1"));
}

@Test
void testDifferentSeeds_ProduceDifferentResults() {
    BloomFilter filter1 = new BloomFilter(1000, 3, 42L);
    BloomFilter filter2 = new BloomFilter(1000, 3, 100L);

    filter1.add("test.ip");
    filter2.add("test.ip");

    // Оба должны найти добавленный IP
    assertTrue(filter1.contains("test.ip"));
    assertTrue(filter2.contains("test.ip"));
}

@Test
void testLargeNumberOfElements() {
```

```
        // Добавляем много элементов
        for (int i = 0; i < 200; i++) {
            filter.add("IP_" + i);
        }
        assertEquals(200, filter.getSize());

        // Проверяем что все находятся (no false negatives)
        for (int i = 0; i < 200; i++) {
            assertTrue(filter.contains("IP_" + i));
        }
    }

    @Test
    void testProbability_GrowsWithElements() {
        double probBefore = filter.getFalsePositiveProbability();

        for (int i = 0; i < 100; i++) {
            filter.add("IP_" + i);
        }

        double probAfter = filter.getFalsePositiveProbability();
        assertTrue(probAfter > probBefore, "Вероятность FP должна расти"
    }
}
```

HashSetFilterTest.java

```
package com.ipfilter.filter;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.*;
import static org.hamcrest.MatcherAssert.assertThat;
```

```
class HashSetFilterTest {

    private IpFilter filter;

    @BeforeEach
    void setUp() {
        filter = new HashSetFilter();
    }

    // --- SPECIFICATION BASED TESTING ---

    // Метод 1: Boundary Value Analysis (BVA)
    @Test
    void testBoundaryValues_EmptyAndSingleElement() {
        // Граница: Пустой фильтр
        assertFalse(filter.contains("192.168.1.1"), "Пустой фильтр не до

        // Граница: Добавление 1 элемента
        filter.add("192.168.1.1");
        assertTrue(filter.contains("192.168.1.1"), "Фильтр должен содерж

        // Граница: Проверка несуществующего рядом с существующим
        assertFalse(filter.contains("192.168.1.2"), "Фильтр не должен со
    }

    // Метод 2: Equivalence Partitioning (EP)
    @ParameterizedTest
    @ValueSource(strings = {"10.0.0.1", "172.16.0.1", "8.8.8.8"})
    void testEquivalencePartitioning_ValidIps(String ip) {
        // Класс эквивалентности: Валидные IP
        filter.add(ip);
        assertTrue(filter.contains(ip));
    }
```

```
@Test
void testEquivalencePartitioning_InvalidInputs() {
    // Класс эквивалентности: Null и пустые строки (не должны ломать
    filter.add(null);
    filter.add("");
    assertEquals(0, filter.getSize(), "Размер не должен измениться п
}

// --- STRUCTURE BASED TESTING ---

// Метод 1: Branch Testing
@Test
void testBranchCoverage_NullInputContains() {
    // Проверяем ветку if (ip == null) внутри contains
    boolean result = filter.contains(null);
    assertFalse(result, "Ветка обработки null должна возвращать fals
}

// Метод 2: Data Flow Testing
@Test
void testDataFlow_DefinitionUse() {
    // Def: создание фильтра
    IpFilter localFilter = new HashSetFilter();
    // Use: добавление
    localFilter.add("1.1.1.1");
    // Use: проверка и получение размера
    int size = localFilter.getSize();
    boolean exists = localFilter.contains("1.1.1.1");

    assertTrue(exists);
    assertEquals(1, size);
}

// --- JUNIT REQUIREMENTS ---
```

```
@Test
void testAssertionsVariety() {
    filter.add("1.1.1.1");
    filter.add("2.2.2.2");

    // 1. assertEquals
    assertEquals(2, filter.getSize());
    // 2. assertTrue
    assertTrue(filter.contains("1.1.1.1"));
    // 3. assertFalse
    assertFalse(filter.contains("3.3.3.3"));
    // 4. assertThrows (проверка на отсутствие исключений при коррек
    assertDoesNotThrow(() -> filter.clear());
    // 5. assertAll (группировка)
    assertAll("Filter State",
            () -> assertEquals(0, filter.getSize()),
            () -> assertFalse(filter.contains("1.1.1.1"))
    );
}

// === НОВЫЕ ТЕСТЫ ДЛЯ MUTATION COVERAGE ===

@Test
void testAdd_NullValue() {
    // Тест на обработку null
    filter.add(null);
    assertEquals(0, filter.getSize(), "Null не должен добавляться");
}

@Test
void testAdd_EmptyString() {
    // Тест на обработку пустой строки
    filter.add("");
    assertEquals(0, filter.getSize(), "Пустая строка не должна добав
}
```

```
@Test
void testAdd_DuplicateValues() {
    // Тест на дубликаты
    filter.add("192.168.1.1");
    filter.add("192.168.1.1");
    assertEquals(1, filter.getSize(), "Дубликаты не должны увеличива
}

@Test
void testContains_NullInput() {
    // Тест ветки if (ip == null)
    assertFalse(filter.contains(null));
}

@Test
void testClear() {
    // Тест метода clear
    filter.add("1.1.1.1");
    filter.add("2.2.2.2");
    filter.clear();
    assertEquals(0, filter.getSize());
    assertFalse(filter.contains("1.1.1.1"));
}

@Test
void testMemoryUsage() {
    // Тест расчёта памяти
    filter.add("1.1.1.1");
    long memoryBefore = filter.getMemoryUsageBytes();
    filter.add("2.2.2.2");
    long memoryAfter = filter.getMemoryUsageBytes();
    assertTrue(memoryAfter > memoryBefore, "Память должна расти");
}
```

```
@Test
void testAdd_Null_DoesNotIncreaseSize() {
    filter.add(null);
    assertEquals(0, filter.getSize(), "Null не должен увеличивать ра
}

@Test
void testAdd_EmptyString_DoesNotIncreaseSize() {
    filter.add("");
    assertEquals(0, filter.getSize(), "Пустая строка не должна увели
}

@Test
void testAdd_Duplicate_DoesNotIncreaseSize() {
    filter.add("192.168.1.1");
    filter.add("192.168.1.1");
    assertEquals(1, filter.getSize(), "Дубликаты не должны увеличива
}

@Test
void testContains_Null_ReturnsFalse() {
    assertFalse(filter.contains(null), "Null должен возвращать false
}

@Test
void testClear_ResetsSize() {
    filter.add("1.1.1.1");
    filter.add("2.2.2.2");
    assertEquals(2, filter.getSize());
    filter.clear();
    assertEquals(0, filter.getSize(), "Clear должен обнулить размер"
    assertFalse(filter.contains("1.1.1.1"), "После clear элементы не
}

@Test
```

```
void testMemoryUsage_GrowsWithSize() {
    long memoryBefore = filter.getMemoryUsageBytes();
    filter.add("1.1.1.1");
    long memoryAfter = filter.getMemoryUsageBytes();
    assertTrue(memoryAfter >= memoryBefore, "Память должна расти или
}

@Test
void testMultipleAdds_DifferentIps() {
    filter.add("1.1.1.1");
    filter.add("2.2.2.2");
    filter.add("3.3.3.3");
    assertEquals(3, filter.getSize());
    assertTrue(filter.contains("1.1.1.1"));
    assertTrue(filter.contains("2.2.2.2"));
    assertTrue(filter.contains("3.3.3.3"));
    assertFalse(filter.contains("4.4.4.4"));
}

@Test
void testMemoryUsage_EmptyFilter() {
    long memory = filter.getMemoryUsageBytes();
    assertEquals(0, memory, "Память пустого фильтра должна быть 0");
}

@Test
void testMemoryUsage_AfterAdd() {
    filter.add("1.1.1.1");
    long memory = filter.getMemoryUsageBytes();
    assertTrue(memory > 0, "Память должна быть > 0 после добавления"
}

@Test
void testSize_AfterMultipleAdds() {
    filter.add("1.1.1.1");
```

```
    filter.add("2.2.2.2");
    filter.add("3.3.3.3");
    assertEquals(3, filter.getSize());
}

@Test
void testSize_AfterClear() {
    filter.add("1.1.1.1");
    filter.clear();
    assertEquals(0, filter.getSize());
}

@Test
void testContains_AfterClear() {
    filter.add("1.1.1.1");
    filter.clear();
    assertFalse(filter.contains("1.1.1.1"));
}

@Test
void testAdd_NullDoesNotIncreaseSize() {
    int sizeBefore = filter.getSize();
    filter.add(null);
    assertEquals(sizeBefore, filter.getSize());
}

@Test
void testAdd_EmptyStringDoesNotIncreaseSize() {
    int sizeBefore = filter.getSize();
    filter.add("");
    assertEquals(sizeBefore, filter.getSize());
}

@Test
void testContains_NullReturnsFalse() {
```

```
        assertFalse(filter.contains(null));
    }

    @Test
    void testClear_MultipleTimes() {
        filter.add("1.1.1.1");
        filter.clear();
        filter.clear(); // Второй вызов не должен ломать
        assertEquals(0, filter.getSize());
    }

    @Test
    void testAdd_SpecialCharacters() {
        filter.add("test@ip#format");
        assertTrue(filter.contains("test@ip#format"));
    }

    @Test
    void testAdd_VeryLongIP() {
        String longIp = "192.168.1.1";
        filter.add(longIp);
        assertTrue(filter.contains(longIp));
    }

    @Test
    void testContains_CaseSensitive() {
        filter.add("192.168.1.1");
        assertFalse(filter.contains("192.168.1.2"));
    }
}
```

FilterServiceTest.java

```
package com.ipfilter.service;

import com.ipfilter.filter.IpFilter;
```

```
import com.ipfilter.model.FilterResult;
import com.ipfilter.model.FilterStatistics;
import com.ipfilter.model.OutputData;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.Assumptions;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import org.junit.jupiter.params.provider.CsvSource;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.*;
import static org.mockito.Mockito.*;

/**
 * Тесты для FilterService с максимальным покрытием мутаций
 *
 * Используемые техники тестирования:
 * - Specification-Based: Boundary Value Analysis, Equivalence Partition
 * - Structure-Based: Branch Testing, Data Flow Testing
 */
class FilterServiceTest {

    private FilterService service;

    @BeforeEach
    void setUp() {
        service = new FilterService();
    }
```

```
// ================================================================
// SPECIFICATION-BASED TESTING
// ================================================================

/**
 * Метод 1: Boundary Value Analysis (BVA)
 * Тестируем граничные значения количества IP
 */
@Test
void testBoundary_EmptyList() {
    // Граница: 0 элементов
    OutputData result = service.processAndCompare(new ArrayList<>())

    assertNotNull(result);
    assertEquals(0, result.getResults().size(), "Пустой список долже
    assertNotNull(result.getTimestamp());
    assertNotNull(result.getStatistics());
    assertNotNull(result.getRecommendation());
}

@Test
void testBoundary_SingleElement() {
    // Граница: 1 элемент
    OutputData result = service.processAndCompare(List.of("192.168.1

    assertEquals(1, result.getResults().size(), "Должен быть 1 резул
    assertNotNull(result.getResults().get(0).getIp());
}

@Test
void testBoundary_ManyElements() {
    // Граница: много элементов
    List<String> ips = new ArrayList<>();
    for (int i = 0; i < 100; i++) {
        ips.add(i + ".0.0." + i);
```

```
    }

    OutputData result = service.processAndCompare(ips);

    assertEquals(100, result.getResults().size());
    assertNotNull(result.getStatistics());
}

/**
 * Метод 2: Equivalence Partitioning (EP)
 * Тестируем классы эквивалентности IP адресов
 */
@ParameterizedTest
@ValueSource(strings = {
        "192.168.1.1",
        "10.0.0.1",
        "172.16.0.1",
        "8.8.8.8",
        "1.1.1.1"
})
void testEquivalence_ValidIpAddresses(String ip) {
    OutputData result = service.processAndCompare(List.of(ip));

    assertNotNull(result);
    assertEquals(1, result.getResults().size());
    assertEquals(ip, result.getResults().get(0).getIp());
}

@ParameterizedTest
@CsvSource({
        "invalid.ip",
        "256.256.256.256",
        "abc.def.ghi.jkl"
})
void testEquivalence_InvalidIpAddresses(String ip) {
```

```
    assertDoesNotThrow(() -> {
        OutputData result = service.processAndCompare(List.of(ip));
        assertNotNull(result);
    });
}

// ============================================================
// STRUCTURE-BASED TESTING
// ============================================================

/**
 * Метод 1: Branch Testing
 * Покрываем все ветви if/else в коде
 */
@Test
void testBranch_ResultSetMatchTrue() {
    // Ветка: оба фильтра возвращают false → match = true
    OutputData result = service.processAndCompare(List.of("8.8.8.8")

    FilterResult filterResult = result.getResults().get(0);
    assertFalse(filterResult.isHashSetResult());
    assertFalse(filterResult.isBloomFilterResult());
    assertTrue(filterResult.isMatch(), "Результаты должны совпадать"
}

@Test
void testBranch_StatisticsFieldsPopulated() {
    // Ветка: заполнение статистики
    OutputData result = service.processAndCompare(List.of("1.1.1.1")
    FilterStatistics stats = result.getStatistics();

    // Проверяем что все поля заполнены и валидны
    assertTrue(stats.getHashSetMemoryBytes() >= 0, "HashSet память д
    assertTrue(stats.getBloomFilterMemoryBytes() >= 0, "Bloom память
    assertTrue(stats.getHashSetTimeMs() >= 0, "HashSet время должно
```

```
    assertTrue(stats.getBloomFilterTimeMs() >= 0, "Bloom время должн
    assertTrue(stats.getFalsePositiveProbability() >= 0, "Вероятност
    assertTrue(stats.getFalsePositiveProbability() <= 1, "Вероятност
}

@Test
void testBranch_RecommendationGenerated() {
    // Ветка: генерация рекомендации
    OutputData result = service.processAndCompare(List.of("1.1.1.1")

    assertNotNull(result.getRecommendation());
    assertFalse(result.getRecommendation().isEmpty());
    assertThat(result.getRecommendation().length(), greaterThan(10))
}

/**
 * Метод 2: Data Flow Testing
 * Отслеживаем определение и использование переменных
 */
@Test
void testDataFlow_OutputDataCreation() {
    // Def: создание OutputData
    OutputData result = service.processAndCompare(List.of("192.168.1

    // Use: проверка всех полей
    assertNotNull(result.getTimestamp(), "Timestamp должен быть опре
    assertNotNull(result.getResults(), "Results должны быть определе
    assertNotNull(result.getStatistics(), "Statistics должны быть оп
    assertNotNull(result.getRecommendation(), "Recommendation должен

    // Use: проверка вложенных объектов
    assertEquals(1, result.getResults().size());
    assertNotNull(result.getResults().get(0).getIp());
}
```

```
@Test
void testDataFlow_FilterResultCreation() {
    // Def: создание FilterResult
    OutputData output = service.processAndCompare(List.of("10.0.0.1"
    FilterResult filterResult = output.getResults().get(0);

    // Use: проверка полей
    assertEquals("10.0.0.1", filterResult.getIp());
    // match вычисляется из hashSetResult и bloomFilterResult
    assertEquals(
            filterResult.isHashSetResult() == filterResult.isBloomFi
            filterResult.isMatch()
    );
}

// ============================================================
// JUNIT 5 REQUIREMENTS
// ============================================================

/**
 * Минимум 5 различных методов assert
 */
@Test
void testAssertionsVariety() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1",

    // 1. assertEquals
    assertEquals(2, result.getResults().size());

    // 2. assertTrue
    assertTrue(result.getResults().size() > 0);

    // 3. assertFalse
    assertFalse(result.getResults().isEmpty());
```

```
    // 4. assertNotNull
    assertNotNull(result.getTimestamp());

    // 5. assertAll (группировка ассертов)
    assertAll("OutputData validation",
            () -> assertNotNull(result.getResults()),
            () -> assertNotNull(result.getStatistics()),
            () -> assertNotNull(result.getRecommendation()),
            () -> assertEquals(2, result.getResults().size())
    );

    // 6. assertDoesNotThrow
    assertDoesNotThrow(() -> service.processAndCompare(List.of("3.3.
}

/**
 * Минимум 2 различных метода assume
 */
@Test
void testAssumptionsUsage() {
    // Assume 1: assumeTrue
    Assumptions.assumeTrue(service != null, "Сервис должен быть созд

    OutputData result = service.processAndCompare(List.of("1.1.1.1")

    // Assume 2: assumingThat
    Assumptions.assumingThat(
            result.getResults().size() > 0,
            () -> {
                assertNotNull(result.getResults().get(0).getIp());
            }
    );

    // Assume 3: assumeFalse
    Assumptions.assumeFalse(result.getResults().isEmpty(), "Результа
```

```
}

/**
 * Parameterized Tests
 */
@ParameterizedTest
@ValueSource(ints = {1, 5, 10, 50})
void testParameterized_DifferentIpCounts(int count) {
    List<String> ips = new ArrayList<>();
    for (int i = 0; i < count; i++) {
        ips.add(i + ".0.0." + i);
    }

    OutputData result = service.processAndCompare(ips);

    assertEquals(count, result.getResults().size());
    assertNotNull(result.getStatistics());
    assertNotNull(result.getRecommendation());
}

// ============================================================
// MOCKITO REQUIREMENTS
// ============================================================

/**
 * Mock Type 1: Standard Mock behavior (when/thenReturn)
 */
@Test
void testMockBehavior_VerifyWithLenient() {
    // Создаём моки для демонстрации
    IpFilter mockFilter = mock(IpFilter.class);

    // Используем lenient() чтобы избежать UnnecessaryStubbingExcept
    lenient().when(mockFilter.contains(anyString())).thenReturn(fals
    lenient().when(mockFilter.getMemoryUsageBytes()).thenReturn(100L
```

```
    lenient().when(mockFilter.getSize()).thenReturn(0);

    // Проверяем что моки работают
    boolean result = mockFilter.contains("1.1.1.1");
    assertFalse(result);

    // Verify вызов
    verify(mockFilter, atLeastOnce()).contains(anyString());
}

/**
 * Mock Type 2: Verify interactions (verify)
 */
@Test
void testMockVerify_Interactions() {
    IpFilter mockFilter = mock(IpFilter.class);

    when(mockFilter.contains("2.2.2.2")).thenReturn(false);
    when(mockFilter.contains("3.3.3.3")).thenReturn(true);

    // Вызываем методы
    mockFilter.contains("2.2.2.2");
    mockFilter.contains("3.3.3.3");

    // Проверяем вызовы
    verify(mockFilter, times(1)).contains("2.2.2.2");
    verify(mockFilter, times(1)).contains("3.3.3.3");
    verify(mockFilter, never()).clear();
}

/**
 * Mock Type 3: Spy (частичный мок)
 */
@Test
void testMockSpy_Usage() {
```

```
    // Создаём spy списка
    List<String> spyList = spy(new ArrayList<>(List.of("A", "B")));

    spyList.add("C");

    // Проверяем что метод был вызван
    verify(spyList).add("C");
    assertEquals(3, spyList.size());

    // Проверяем оригинальное поведение
    assertEquals("A", spyList.get(0));
    assertEquals("B", spyList.get(1));
    assertEquals("C", spyList.get(2));
}

/**
 * Mock Type 4: Exception testing with mocks
 */
@Test
void testMockException_Throwing() throws IOException {
    JsonHandler mockHandler = mock(JsonHandler.class);

    // Настраиваем исключение
    doThrow(new IOException("File not found"))
            .when(mockHandler).loadInput(anyString());

    // Проверяем что исключение выбрасывается
    assertThrows(IOException.class, () -> {
        mockHandler.loadInput("nonexistent.json");
    });
}

// ================================================================
// HAMCREST MATCHERS
// ================================================================
```

```
/**
 * Минимум 2 различных вида Hamcrest matchers
 */
@Test
void testHamcrestMatchers() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")
    FilterStatistics stats = result.getStatistics();

    // Matcher 1: greaterThan
    assertThat(stats.getHashSetMemoryBytes(), greaterThanOrEqualTo(0

    // Matcher 2: lessThan
    assertThat(stats.getFalsePositiveProbability(), lessThanOrEqualT

    // Matcher 3: equalTo
    assertThat(result.getResults().size(), equalTo(1));

    // Matcher 4: notNullValue
    assertThat(result.getTimestamp(), notNullValue());

    // Matcher 5: containsString (для recommendation)
    assertThat(result.getRecommendation(), not(emptyString()));
}

// ============================================================
// ADDITIONAL TESTS FOR MUTATION SCORE
// ============================================================

@Test
void testTimestamp_Format() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")

    String timestamp = result.getTimestamp();
    assertNotNull(timestamp);
```

```
    // Проверяем формат ISO (должен содержать дату и время)
    assertTrue(timestamp.length() >= 19, "Timestamp должен быть в фо
    assertTrue(timestamp.contains("-") || timestamp.contains("T"),
            "Timestamp должен содержать разделители даты/времени");
}

@Test
void testResult_IPField() {
    String testIp = "192.168.100.1";
    OutputData result = service.processAndCompare(List.of(testIp));

    assertEquals(testIp, result.getResults().get(0).getIp());
}

@Test
void testStatistics_FalsePositiveProbability() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")
    FilterStatistics stats = result.getStatistics();

    assertTrue(stats.getFalsePositiveProbability() >= 0);
    assertTrue(stats.getFalsePositiveProbability() <= 1);
}

@Test
void testStatistics_TimeNonNegative() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")
    FilterStatistics stats = result.getStatistics();

    assertTrue(stats.getHashSetTimeMs() >= 0);
    assertTrue(stats.getBloomFilterTimeMs() >= 0);
}

@Test
void testStatistics_MemoryComparison() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")
```

```
    FilterStatistics stats = result.getStatistics();

    // Оба значения памяти должны быть положительными
    assertTrue(stats.getHashSetMemoryBytes() >= 0);
    assertTrue(stats.getBloomFilterMemoryBytes() > 0);
}

@Test
void testServiceConstructor_Default() {
    FilterService newService = new FilterService();
    assertNotNull(newService);
}

@Test
void testProcessAndCompare_SingleIp() {
    OutputData result = service.processAndCompare(List.of("8.8.8.8")

    assertEquals(1, result.getResults().size());
    assertNotNull(result.getStatistics());
    assertNotNull(result.getRecommendation());
    assertFalse(result.getRecommendation().isEmpty());
}

@Test
void testProcessAndCompare_MultipleIps() {
    List<String> ips = List.of("1.1.1.1", "2.2.2.2", "3.3.3.3");
    OutputData result = service.processAndCompare(ips);

    assertEquals(3, result.getResults().size());

    // Проверяем каждый результат
    for (FilterResult fr : result.getResults()) {
        assertNotNull(fr.getIp());
        assertNotNull(fr.isMatch());
    }
```

```
}

@Test
void testProcessAndCompare_StatisticsPopulated() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")
    FilterStatistics stats = result.getStatistics();

    assertTrue(stats.getHashSetMemoryBytes() >= 0);
    assertTrue(stats.getBloomFilterMemoryBytes() >= 0);
    assertTrue(stats.getHashSetTimeMs() >= 0);
    assertTrue(stats.getBloomFilterTimeMs() >= 0);
    assertTrue(stats.getFalsePositiveProbability() >= 0);
}

@Test
void testProcessAndCompare_ResultMatch() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")
    FilterResult filterResult = result.getResults().get(0);

    // Для IP не в blacklist оба должны вернуть false
    assertFalse(filterResult.isHashSetResult());
    assertFalse(filterResult.isBloomFilterResult());
    // match должен быть true (оба false)
    assertTrue(filterResult.isMatch());
}

@Test
void testOutputData_AllFieldsNotNull() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")

    assertAll("All fields not null",
            () -> assertNotNull(result.getTimestamp()),
            () -> assertNotNull(result.getResults()),
            () -> assertNotNull(result.getStatistics()),
            () -> assertNotNull(result.getRecommendation())
```

```
    );
}

@Test
void testFilterResult_AllFields() {
    OutputData result = service.processAndCompare(List.of("8.8.8.8")
    FilterResult filterResult = result.getResults().get(0);

    assertAll("FilterResult fields",
            () -> assertNotNull(filterResult.getIp()),
            () -> assertTrue(filterResult.isHashSetResult() || !filt
            () -> assertTrue(filterResult.isBloomFilterResult() || !
            () -> assertTrue(filterResult.isMatch() || !filterResult
    );
}

@Test
void testResult_MatchField() {
    OutputData result = service.processAndCompare(List.of("1.1.1.1")
    FilterResult filterResult = result.getResults().get(0);

    // match должен быть true когда результаты совпадают
    boolean expectedMatch = (filterResult.isHashSetResult() == filte
    assertEquals(expectedMatch, filterResult.isMatch());
}

@Test
void testProcessAndCompare_DuplicateIPs() {
    // Тестируем дубликаты IP
    OutputData result = service.processAndCompare(
            List.of("1.1.1.1", "1.1.1.1", "1.1.1.1")
    );

    // Должно обработать все 3 IP (даже дубликаты)
    assertEquals(3, result.getResults().size());
```

```
}

@Test
void testProcessAndCompare_EmptyString() {
    // Тестируем обработку пустой строки
    OutputData result = service.processAndCompare(List.of(""));

    assertNotNull(result);
    assertEquals(1, result.getResults().size());
}

@Test
void testProcessAndCompare_NullIP() {
    // Тестируем обработку null IP
    List<String> ips = new ArrayList<>();
    ips.add(null);

    assertDoesNotThrow(() -> {
        OutputData result = service.processAndCompare(ips);
        assertNotNull(result);
        assertEquals(1, result.getResults().size());
    });
}

@Test
void testRecommendation_MemoryDifference() {
    OutputData result = service.processAndCompare(
            List.of("1.1.1.1", "2.2.2.2", "3.3.3.3", "4.4.4.4", "5.5
    );

    assertNotNull(result.getRecommendation());
    assertTrue(result.getRecommendation().length() > 0);
}

@Test
```

```
void testRecommendation_LargeDataSet() {
    List<String> ips = new ArrayList<>();
    for (int i = 0; i < 50; i++) {
        ips.add(i + "." + i + "." + i + "." + i);
    }

    OutputData result = service.processAndCompare(ips);

    assertNotNull(result.getRecommendation());
    assertFalse(result.getRecommendation().isEmpty());
}

@Test
void testProcessAndCompare_MixedResults() {
    OutputData result = service.processAndCompare(
            List.of("1.1.1.1", "2.2.2.2", "3.3.3.3")
    );

    assertEquals(3, result.getResults().size());

    // Проверяем что все результаты имеют IP
    for (FilterResult fr : result.getResults()) {
        assertNotNull(fr.getIp());
    }
}

@Test
void testProcessAndCompare_InvalidIP() {
    assertDoesNotThrow(() -> {
        OutputData result = service.processAndCompare(List.of("inval
        assertNotNull(result);
        assertEquals(1, result.getResults().size());
    });
}
```

```
    @Test
    void testServiceConstructor_WithMocks() {
        // Тест конструктора с моками
        IpFilter mockHashSet = mock(IpFilter.class);
        IpFilter mockBloom = mock(IpFilter.class);
        JsonHandler mockJsonHandler = mock(JsonHandler.class);

        FilterService serviceWithMocks = new FilterService(mockHashSet,
        assertNotNull(serviceWithMocks);
    }
}
```